

ตาม (Tam)

a cat that reads the standup you did not have

DRAFT DECK · 10 สไลด์

18 ส.ค. 2569 · ส่ง 19 ส.ค. 2569



01

ปก

ตาม — สมุดบัญชีงานที่อยู่ใน Slack

รู้ว่าจะไรตีด ก่อนเข้า standup — และทุกคำที่ระบบพูด ถอดกลับไปดูข้อความที่เป็นหลักฐานได้ในคลิกเดียว

3

วิธีการใช้งาน: CLI · Dashboard ·
Slack bot

100%

สถานะงานคำนวณด้วยกฎ ไม่ใช่ AI เดา

0

ข้อมูลที่ออกนอกเครื่อง (ค่า default)

ต้นแบบใช้งานได้จริงแล้วทั้งสามผิว โค้ดทั้งหมดเปิดอยู่ที่ github.com/SirawithPra/scraping-slack-data

รอบนี้เลิกวัดบนข้อมูลตัวอย่าง — export จริง 4 ช่องภายใน 1,057 ข้อความ ถูกใช้เป็นชุดวัดผล ทำให้ทั้งโมเดลและเกณฑ์ตัดความเกี่ยวข้อง**เปลี่ยนเพราะตัวเลข ไม่ใช่เพราะความรู้สึก** · เผยแพร่เฉพาะตัวเลขรวม
เนื้อหาข้อความไม่ออกจากเครื่อง



02 · READING THE CHANNEL

สถานะจริงของงานอยู่ใน 4 ที่ และ ครบอยู่ที่เดียว — ในหัวคน

อาการที่ทีมเจอดูเหมือนหลายเรื่อง แต่เป็นเรื่องเดียวกันในเลี้ยวคนละชุด

เช้าวันจันทร์

เข้า standup แล้วต้องนึกเองว่าใครติดอะไรไว้
เมื่อวันศุกร์

งานที่หายไป

เรื่องที่คุยกันในเรดแล้วไม่มีใครเปิด ticket ก็
หายไปเลย

REQUIREMENT เปลี่ยน

ตกลงกันใหม่ใน Slack แต่ ticket ยังเป็นของ
เดือนพฤษภาคม

ถามซ้ำ

"เรื่องนี้สรุปว่าจะไร" ถูกถามใหม่ทุกสอง
สัปดาห์ เพราะไม่มีที่ให้ย้อนดู

เครื่องมือที่มีอยู่แต่ไม่ตรงจุด — bot รวบรวม standup ได้แต่ต้องให้คนพิมพ์เอง, dashboard สวยแต่
ข้อมูลต้องกรอก, ticket system รู้แต่เรื่องที่มีคนเปิด ticket

อ้างอิง docs/DESIGN_BRIEF.md

ขอบเขต ทีมซอฟต์แวร์ที่คุยงานใน Slack เป็นหลัก

รวมเป็น "งานหนึ่งชิ้น" แล้วส่งให้ตอนที่คุณกำลังใช้ความจำ

“พืชนั่นเจอว่าคะแนนมันเปลี่ยนไปเปลี่ยนมา...”
 “ในรูป record จาก prod ะ”
 “สรุปแล้วเดี๋ยวพืชนั้นไปถาม user อีกที”
 “mkt ต้องรอ เดี่ยวเป็นสปีนใหม่แยก”
 • 22 ข้อความ • 3 ช่อง • 41 วัน



PROJ-225 • uat, event mobile preview
 state **blocked** since 2026-08-18
 evidence “mkt ต้องรอ ...” ↗
 slack 22 • meeting 5 • ticket 2
 waiting on **mkt**

ซ้ายคือสิ่งที่มืออยู่จริงใน Slack ขวาคือสิ่งที่ระบบสรุป — ทุกบรรทัดขวาถูกกลับไปดูข้อความต้นทางได้

ไม่ใช่ dashboard เพิ่มอีกอัน — เป็นระบบที่อ่านสิ่งที่ทีมพิมพ์กันอยู่แล้ว แล้วเก็บเป็นบันทึกเดียวต่อหนึ่งงาน

ข้อความ Slack + เทรด + บันทึกการประชุม (VTT / SRT)

→ ทำความสะอาด กรอง noise ให้เป็น record รูปแบบเดียวกัน

→ จับกลุ่มเป็น work item • คำนวณสถานะ • ผูกหลักฐานทุกข้อ

→ อ่านได้ 3 ทาง: CLI • Dashboard • ในหน้าต่าง Slack เอง

จุดที่ต่างจริง คือการประชุมไม่ได้เป็น pipeline ที่สอง — บันทึกการประชุมถูกแปลงเป็น record ชนิดเดียวกับ Slack ทำให้งานหนึ่งชิ้นข้ามสองแหล่งได้โดยไม่ต้องเขียนโค้ดใหม่

ตัวอย่างที่ commit ไว้ `standup.vtt` → 9 record ชนิดเดียวกับ Slack ในไฟล์เดียวกัน (27 record รวม)

ไม่มี pipeline ที่สอง `tam.ingest.meetings --merge-into` เขียนลง corpus เดิม

“ทำไมไม่ต่อ Claude เข้า Slack แล้วถามมันเลย”

คำถามนี้ถูก และต้องตอบก่อนอย่างอื่น — Slack, Notion, Jira มี connector พร้อมใช้วันนี้ ถามว่า “อะไรติดอยู่” แล้วมันตอบได้จริง

เราจึงไม่ขายว่า “อ่าน Slack แล้วสรุปได้” — ข้อนั้นแพะ และแพะเพราะมันจริง สิ่งที่มีผู้ช่วยแบบถาม-ตอบทำไมได้ ไม่ใช่ทำได้แย่กว่า มีสามข้อ

01 · ไม่มีใครถาม

คุณค่าเกิด 09:25 ก่อน standup และ 08:45 ใน DM อาการของปัญหาคือ “ลืมนึกถึง” — เครื่องมือที่ต้องเรียกใช้ แก้ปัญหาที่อาการคือการลืมนึกใช้ไม่ได้

02 · ถามซ้ำได้คำตอบเดิม

ถาม LLM สองครั้งอาจได้สถานะไม่เท่ากัน standup ต้องการคำตอบที่ทำซ้ำได้สถานะที่นี้คำนวณด้วยกฎไม่ผ่านโมเดล

03 · เฝ้าโดยไม่มีใครสั่ง

ไม่มีใครเปิดมาถามว่า “สัปดาห์นี้ ticket ไหนในหลายสิบใบ ไม่ตรงกับที่คุยใน Slack” การเฝ้าต้องยืนอยู่ตลอด

กลไกที่ทำให้ข้อ 02 เป็นจริง

คำนวณด้วยกฎ · ตรวจได้

งานชิ้นไหน ใครเกี่ยวข้อง ติดหรือไม่ ตั้งแต่เมื่อไหร่ และข้อความไหนเป็นหลักฐาน

✓ derived

ให้โมเดลเขียน · ติดป้ายไว้

เฉพาะประโยคสรุป ต้องอ้าง message id ที่มีจริง id ที่ไม่มีในงานนั้นถูกตัดออก เหลือศูนย์ก็ติดป้าย **unverified**

⚠ generated

เราไม่ได้แข่งกับ Claude — เราใช้มัน ระบบมี `SUMMARIZER=claude` อยู่แล้วสำหรับเขียนภาษา แต่สถานะไม่เคยผ่านมัน จุดต่างจึงไม่ใช่ความฉลาด แต่คือความรับผิดชอบต่อคำพูด และการทำงานเองโดยไม่มีใครสั่ง

ประโยชน์เดียว ผู้ช่วยตอบเมื่อคุณถาม และคุณต้องเชื่อคำตอบ · อันนี้เฝ้าเอง และถูกบรรทัดบอกได้ว่าข้อความไหนเป็นหลักฐาน

สามผิว หนึ่งแนวคิด แยกกันคนละ process

ผิว	ใช้ตอนไหน	เทคโนโลยี	เริ่มด้วย
CLI	พัฒนาและวัดผลตัวระบบ	Python	<code>python3 -m tam.retrieval.retrieve</code>
Dashboard	อ่านว่าวันนี้ทีมติดอะไร	FastAPI	<code>python3 -m tam.web.server</code>
Slack bot	ในที่ทำงานเกิดขึ้นจริง	TypeScript · Bolt	<code>npm start</code>

แยกกันเป็นคนละ process โดยตั้งใจ — บอกลุ่มไม่ทำให้ dashboard ล่ม และกลับกัน

```
tam/      แพ็กเกจ Python – 31 โมดูล (26 เป็น CLI), python3 -m tam.<area>.<m
├─ ingest/   ดึง Slack · ทำความสะอาด · บันทึกการประชุม
├─ retrieval/ embeddings · BM25 · signals · fusion · rerank
├─ analysis/ graph · relations · linker · digest · summarize
├─ evaluation/ วัดผล recall/nDCG · fine-tune
├─ report/   visualize (Plotly) · report_th (ภาษาไทยล้วน)
└─ web/      server.py – dashboard

slack-bot/  Slack bot – npm project แยก
```

โครงเดิม 25 ไฟล์ .py กองที่ root

ปัจจุบัน root ไม่มีไฟล์ .py เลย



03 · ON THE DASHBOARD

รับบนข้อมูลจริงอยู่ตอนนี้ — และ โมเดลถูกเปลี่ยนเพราะการวัด

```
$ curl -s http://127.0.0.1:8899/api/health # ตัดฟิลต์เวลาออกให้อ่านง่าย
{"ok":true,"records_path":"data/processed/real_all.json","records":1301,"topic
"model":"BAAI/bge-m3","summariser":"template","preset":"hybrid","window_days"

$ curl -s http://127.0.0.1:8899/api/blockers # ชื่อถูกแทนด้วยนามสมมุติ
{"blocked":[
  {"label":"cms, api, format ka","evidence":"blocked since 2026-06-23 -
    คนกรอกเองว่า \"waiting for clearing user on dev because data issues\""},
  {"label":"uat","evidence":"blocked since 2026-08-18 - cue \"ต้องรอ\""}]}
```

63.7%

triplet accuracy ของ **bge-m3** บน
corpus จริง · โมเดลเดิม (MiniLM) ได้
59.4%

0.731

cosine สูงสุดที่ query ขยะทำได้ · เดิม
0.918 — ยิ่งต่ำยิ่งดี

8192

context token · เดิม 128 จึงตัด
ข้อความจริงทิ้งเจียบ ๆ 13.4% ของ
record

โมเดลที่เราเทรนเองสองตัวแพ้ของสำเร็จรูป — ได้ 61.4% และ 60.9% เทียบ 63.7% และทั้งคู่ดันให้ query
ขยะได้ cosine **สูงขึ้น** (0.971 · 0.964) จึงไม่ได้ใช้ทั้งสองตัว นี่คือผลการวัด ไม่ใช่ความล้มเหลว: corpus
ระดับนี้เล็กลงกว่าจะชนะของสำเร็จรูป ตารางเดิมเจ็ดโมเดลอยู่ใน [docs/EXPERIMENTS.md](#) ที่เดียว เพื่อไม่ให้
ตัวเลขสองที่เพี้ยนกัน

8 หน้าจอ

`/digest` · `/blockers` · `/people`
ใครอยู่กับเรื่องไหน · `/person/{user}`
คนหนึ่งคนแบบละเอียด · `/tracker` เกี่ยวกับ
ticket · `/item/{key}` timeline ·
`/search` หาเส้นทางของประโยค ·
`/upload` วางโน้ตหรือไฟล์ประชุม

7 API + HEALTH

ข้อมูลชุดเดียวกันออกเป็น JSON —
`/api/digest` · `/api/blockers` ·
`/api/people` · `/api/tracker` ·
`/api/item/{key}` · `/api/search` ·
`/api/health` และที่เขียนข้อมูล `POST`
`/api/reindex` · `POST /upload` ·
`POST /upload/notes` ที่ต้องมี token

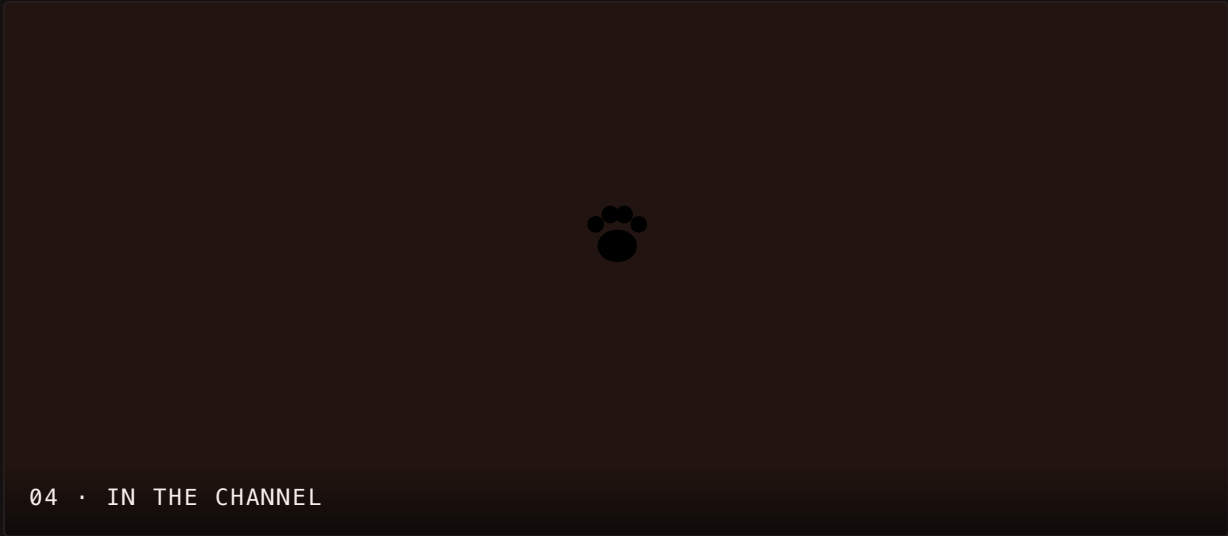
✓ ทดสอบแล้ว ยังทั้ง **16 endpoint** ที่อ่านข้อมูล (8 หน้า + 8 API) คืบ **200** พร้อมเนื้อหาจริงทุกตัว · อีก
3 route ที่เขียนข้อมูล (`POST /upload` , `POST /upload/notes` , `POST /api/reindex`) ต้อง
แบบ token ที่ server พิมพ์ตอน start

⚠ อ่านตัวเลขนี้ให้ตรง `blocked` ได้ **7 จาก 38 งาน** — และตัวเลขนี้ขึ้นมาจาก **2** เพราะแก็กล็อก ไม่ใช่เพราะ
ปรับเกณฑ์: เดิมการประกาศว่าติดของคนหนึ่งถูกยกเลิกโดย cue “closed” ที่ไหนก็ได้ใน cluster เดียวกัน ทั้ง
ที่ cluster นั้นกว้าง 71 ข้อความ 69 วัน **ตอนนี้มีแต่เจ้าตัวเท่านั้นที่ยกเลิกคำประกาศของตัวเองได้** และ cue
ถูกอ่านทีละ **ussกั๊ด** ไม่ใช่ทั้งโพสต์ · ยังต่ำกว่าความจริงอยู่ (ดูสไลด์ 10)

ไม่มี build step HTML สร้างจาก Python ตรง ๆ ไม่มี
`node_modules`

ไม่มี Slack ก็รับได้ `data/sample/` commit ไว้แล้ว

log สะอาด warning เกือบจาก BLAS ปิดเฉพาะจุด — ตรวจกับ float64 แล้วว่าตรงกันถึง 5e-08



ใช้งานในหน้าต่าง Slack ไม่ต้องเปิดเว็บ

ใช้ **Socket Mode** — ไม่ต้องมี public URL ไม่ต้องใช้ ngrok ไม่ต้องเปิด firewall บอกต่อออกไปหา Slack เอง รับจากโน้ตบุ๊กได้เลย

พิมพ์ใน SLACK	ทำอะไร
<code>/meowtam</code> · <code>/mt</code>	บอร์ดงานทั้งหมด เรียงงานที่ติดขึ้นก่อน
<code>/meowtam blocked</code>	เฉพาะที่ติด พร้อมข้อความที่เป็นหลักฐาน
<code>/meowtam recall</code> <ข้อความ>	ค้นหาไทยได้ พร้อมสายการตัดสินใจของเรื่องนั้น — ผ่าน gate สองสัญญาณก่อนตอบ
<code>/meowtam MOB-142</code>	งานชิ้นเดียวตาม key — key นี้มีอยู่ใน ledger ตัวอย่างที่ commit ไว้
คลิกขวาที่ข้อความ	ผูกกับ ticket · บันทึกเป็นการตัดสินใจ
ใส่ emoji	<code>reaction_added</code> เป็น input — emoji สั่งงานได้

8

work item ใน ledger ตัวอย่าง

128

ข้อความใน corpus ตัวอย่างที่ ledger นั้นสร้างมาจาก

8

หน้าจอรุ่นเดอร้อฟไลน์ — ทุก payload ผ่าน Slack limits

✓ ทดสอบแล้ว

`npm install` · `npm run typecheck` ไม่มี error · `npm test` ผ่าน **120/120**

(ในนั้นมี `tests/gate.test.ts` 6 ข้อ ที่คุณเกณฑ์ตัดความเกี่ยวข้อง และ

`tests/postguard.test.ts` 8 ข้อ ที่คุณว่าโพสต์ออก Slack ได้เฉพาะปลายทางที่อนุญาต ไว้ไม่ให้เพื่อน

เจียบ ะ) · `npm run preview` เรนเดอร์ทั้ง 8 หน้าจอบวกเคสที่ยาวสุดของสี่หน้าที่มีลิสต์ ออฟไลน์ ผ่าน
ทั้งหมด

DM 08:45 · digest 09:25 ปิดไว้เป็น default เปิดด้วย ENABLE_SCHEDULE=1

ข้อมูลไม่ออกนอกเครื่อง — และนี่คือประตู ไม่ใช่ความชอบ

อีกสองจุดต่างที่ **connector** ผ่านไม่ได้ ค่า default ของระบบนี้ไม่ส่งข้อความออกนอกเครื่องเลย โมเดลทุกตัวรันในเครื่อง — สำหรับองค์กรที่ไม่อนุญาตให้ Slack ไล่ออก API ต่างประเทศ นี่เป็นเงื่อนไขผ่าน/ไม่ผ่าน และภาษาไทยที่วัดผลได้ ไทยไม่มีวันวรรณ เราจึงใช้ Thai BM25 + character n-gram + multilingual embeddings และมีเลข recall/nDCG กำกับ ไม่ใช่ keyword search ที่เสื่อมเจียบๆ

ริโปเป็น public จึงต้องแยกให้ชัดว่าจะโผล่แพร่ได้ อะไรไม่ได้ — และตรวจจริง ไม่ใช่ตั้งใจไว้

สิ่งของ	อยู่ที่ไหน	ขึ้น GIT
token ทั้งหมด	<code>.env</code> , <code>slack-bot/.env</code>	✗ ไม่
export จริงจาก Slack	<code>data/raw/</code>	✗ ไม่
records + embedding cache	<code>data/processed/</code>	✗ ไม่
โมเดลที่เทรนเอง 2 ตัว (465 MB ต่อตัว) — วัดแล้วไม่ผ่าน ไม่ได้ใช้	<code>models/</code>	✗ ไม่
โมเดล embedding ที่เสิร์ฟจริง (BAAI/bge-m3)	cache ของ Hugging Face นอกชิป · โหลดครั้ง แรกครั้งเดียว	✗ ไม่
ledger ที่ <code>npm run ledger</code> เขียนจาก export จริง	<code>slack-bot/data/ledger.json</code>	✗ ไม่
ข้อมูลตัวอย่างสังเคราะห์ + ledger ตัวอย่าง	<code>data/sample/</code> · <code>ledger.fixture.json</code>	✓ ตั้งใจ

- 🔥 **ไฟล์ที่ commit อยู่ตอนนี้ไม่มี Slack ID จริงเลย** — เป็น placeholder ที่คงความยาวเดิมไว้ให้ parser ไม่พัง (`C0SAMPLE01` , `C0DEMOCHAN1` , `U0DEMOUSER1`) ชื่อคน ชื่อบริษัท ชื่อโปรเจกต์ภายใน เปลี่ยนเป็นชื่อสมมติทั้งหมด
- 🔥 **แต่ประวัติ git ยังไม่สะอาด — พุดตรงๆ** commit แรก (`8c27553`) ยังมี ID จริงอยู่ก่อนสอง commit ที่มาแทนที่ทีหลัง ใน repo public นั้นถึงตาม SHA ได้ ต้อง rewrite history แล้ว force-push จึงจะหมดจริง — ยังไม่ได้ทำ ไม่ใช่ credential (token ไม่เคยหลุด) แต่ก็ยังเป็นงานที่ค้าง
- 🔥 **ไม่มี token ในโค้ดเลย** — อ่านจาก env ทุกจุด และ `.env.example` อธิบายครบทุกตัวแปร · route ที่เขียนข้อมูลบน dashboard ต้องมี token ที่สร้างใหม่ทุกครั้งที่ start และปฏิเสธ `Origin` ข้ามเว็บ

ตัวอย่าง id ที่เผยแพร่

`msg_C0SAMPLE01_1754000010.000100`

สัญญาอนุญาต MIT

ของที่ต้องส่ง 19 ส.ค. 2569

รายการ	สถานะ	หลักฐาน
Working Prototype	✓ พร้อม	Dashboard 16 endpoint ที่อ่านข้อมูลคืน 200 · bot typecheck สะอาด, <code>npm test</code> ผ่าน 120/120, ทุก payload ผ่าน Slack limits · 26 CLI Python ตอบ <code>--help</code> คนทุกตัว และ <code>pytest</code> ผ่าน 178/178
User Manual	✓ พร้อม	<code>docs/USER_MANUAL.md</code> — ติดตั้ง ใช้งาน ต่อ Slack แก้ปัญหา ทุกคำสั่งรับผ่านจริง
Draft Presentation	✓ พร้อม	สไลด์นี้ 10 แผ่น · ตัวเลขทุกตัวมีบันทึกการวัดอยู่ใน docs/EXPERIMENTS.md
Source Code	✓ พร้อม	github.com/SirawithPra/scraping-slack-data — public

ทดลองเองได้โดยไม่ต้องมี Slack token — รีโปมีข้อมูลตัวอย่างไทย/อังกฤษ commit ไว้ ทำงานครบทุกขั้น ตั้งแต่ ingest ถึง dashboard

```
git clone https://github.com/SirawithPra/scraping-slack-data.git
cd scraping-slack-data && python3 -m venv .venv && source .venv/bin/activate
cd pipeline && python3 -m pip install -r requirements.txt

python3 -m tam.ingest.prepare_messages --raw data/sample/slack_messages.sample
--out data/processed/sample_combined.json
python3 -m tam.ingest.meetings --transcript data/sample/standup.vtt \
--title "Daily standup" --started 2026-08-14T09:30 \
--merge-into data/processed/sample_combined.json
python3 -m tam.web.server --records data/processed/sample_combined.json --days
```

ทุกคำสั่งรับจาก `pipeline/` เพราะแพ็กเกจ `tam` อยู่ที่นั่น · โฟล์ `data/processed/` ไม่ขึ้น git จึงต้องสร้างจากข้อมูลตัวอย่างสองคำสั่งแรก · `--days 3650` เพราะ export ตัวอย่างลงวันที่ 2025-08-01 หน้าต่าง default 7 วันจะว่าง · `EMBEDDING_MODEL` ใน `.env.example` ปลอ่ยว่างไว้ ให้ค่า default ในโค้ด (`BAAI/bge-m3`) เป็นตัวตัดสินใจ จะได้ไม่มีสองแหล่งความจริง

```
INFO Building index from data/processed/sample_combined.json
INFO Embedding 27 unique text(s); 0 already cached
INFO Loading embedding model BAAI/bge-m3 (the first run downloads it)
INFO Ready: 29 record(s), 11 topic(s), 4 blocked, summariser template
```

เวลาติดตั้ง ~2 นาที + ดาวน์โหลดโมเดลรอบแรก

ทำอะไร Ready: 29 record · 11 topic · 4 blocked

ทดสอบบน macOS · Python 3.10.17 · Node v24.3.0



05 · WHAT IS LEFT

เกณฑ์ตัดความเกี่ยวข้องกรองผิด
สัญญาณมาตลอด — การวัดคือสิ่งที่จับได้

cosine สูงสุดไม่ได้บอกว่า “มีของให้ตอบ” — มันบอกว่า corpus ใหญ่

gate เดิมถามข้อเดียว: cosine ของ record ที่ใกล้สุดถึงเกณฑ์ไหม แต่ `max cosine` เหนือ N เอกสารสูงขึ้นตาม N สำหรับ query อะไรก็ตาม — มี ~1,000 record ยังไงก็มีอันที่ถูกคล้าย สไลด์นี้เวอร์ชันก่อนเขียนว่า MiniLM แยกขยะออกได้และตัวที่ fine-tune เองเป็นตัวที่พัง **ข้อนั้นผิด**: วัดบน corpus จริงแล้ว **ไม่มีโมเดลไหนในห้าตัวที่มี floor ซึ่งแยกได้** MiniLM รวมอยู่ในนั้นด้วย

กลไกที่ลอง	ขยะที่กรองได้	QUERY จริงที่เสียไป
<code>cosine ≥ floor</code> เดียว (ของเดิม)	0/5 — ทั้งห้าโมเดล	0/12
cross-encoder reranker เป็น gate	ปฏิเสธ — คำถามไทยจริงได้ 0.008 ขณะที่ขยะได้ 0.114	—
<code>BM25 > 0</code> และ <code>cosine ≥ floor</code> (ใช้ตัวนี้)	4/5	0/12

ทำไมสัญญาณคำถึงชนะ: ขยะไม่มีคำตรงกับ corpus เลย BM25 จึงได้ `0.000` สนิท ขณะที่ cosine หาอะไรคล้ายได้เสมอ — บน corpus ตัวอย่าง 27 record ที่ commit ไว้ query ขยะได้ `bm25 0.00 · cos 0.486` คือผ่านเกณฑ์ cosine 0.45 แต่ตกที่สัญญาณคำ ส่วน dense ยังต้องมียู่เพื่อจับคำถามจริงที่ใช้คำต่างจาก corpus · `/api/search` จึงคืน `relevance: {lexical, dense}` เป็นค่าดิบทั้งคู่ และ pipeline ที่เก่าเกินจะส่งฟาล์วนี้ **throw ทั้ง ไม่ปล่อยผ่านเจียบ ๆ**

⚠ ขอบเขตที่ยังอยู่ ตัวที่ยังหลุดคืออักขระไทยซ้ำ ๆ (ๆ ๆ) ซึ่งมีอยู่ในข้อความปกติจริง จึงมีคำตรงกันจริง — `check-api` เก็บ probe ตัวนี้ไว้ในลิสต์**ตั้งใจ** และรายงาน 3/4 แทนที่จะซ่อนให้ดูสวย · บน corpus จริงที่รันอยู่: กรองขยะ **3/4** เสีย query จริง **0/6** · บน corpus ตัวอย่าง 27 record มันกรองขยะได้ 4/4 แต่ตัด probe จริงทิ้ง 1 อัน ซึ่งเป็น label ที่ไม่มีคำร่วมกับ corpus เลย (`no shared anchor`) ไม่ใช่คำถามที่คนพิมพ์ — corpus เล็กก็เห็นทั้งสองด้านของกลไกชัดกว่า

ปิดไปแล้ว

บอกกับ pipeline นับ “งานหนึ่งชิ้น” เหมือนกันแล้ว

เดิมสองฝั่งอ่าน Slack เองและจับกลุ่มด้วยกฎต่างกัน ช่องเดียวกันจึงได้ work item ไม่เท่ากัน ตอนนี้ตั้ง `TAM_API_URL` แล้วบอกเลิกตัดสินใจเอง — work item, สถานะ, หลักฐาน, timeline และ recall มาจาก pipeline ทั้งหมด

✓ ทดสอบแล้ว `npm run check-api` บนข้อมูลจริง: **38 work item** · หลักฐาน, citation และ timeline ทุกอันชี้ข้อความที่มีจริงใน item นั้น · permalink สร้างได้ **642/691** ข้อความที่มาจาก Slack (ที่เหลื่อมมาจากไฟล์ประชุมและทิกเก็ต จึงไม่มี permalink ตามคาด) · standup draft 16 อันคำนวณจาก item ที่ได้มา ส่วน decision 0 เพราะยังไม่มีคนกดบันทึก — **ไม่เติมให้ดูเต็ม** · exit code ตอบแล้วว่า “รอยต่อนี้ถูกไหม” (ได้ 0) การ calibrate เกณฑ์รายงานให้เห็นแต่ไม่ทำให้ fail เว้นใส่ `--strict-gate` (แล้วได้ 1) เพราะ floor เป็นคุณสมบัติของ corpus คู่กับโมเดล ไม่ใช่ของรอยต่อ

ปิดไปแล้ว

ต่อ ticket system จริงแล้ว — และแหล่งที่สองเจอสิ่งที่ Slack เจียบสนิก

ticket อยู่ใน corpus เดียวกันกับ Slack แล้ว ไม่ใช่แค่เอามาเทียบ — ค้นครั้งเดียวเจอทั้งสองแหล่ง
สิ่งที่ embed คือชื่อ ticket บวก 2 บรรทัดแรกของ description ที่มีเนื้อหาจริง (ไม่ใช่ทั้งก่อน:
median ของ Slack อยู่ที่ 46 ตัวอักษร ของ ticket 582 และมีใบหนึ่ง 14,556 ซึ่งเป็นตาราง test
case ล้วน) และ ticket ห้ามตั้งตัวเป็นงานเอง — วัดแล้ว 43 จาก 196 ใบ ไปเกาะกับบทสนทนาจริง
ใน 19 งาน ส่วนที่เหลือไม่ตั้งตัวเป็นงานเลย (0 งานที่มีแต่ทิกเก็ต) แอดยังค้นเจอครบ อ่าน ticket จริง
197 ใบ (เปิดค้าง 57) มาเทียบกับที่คุยใน Slack ผลที่ได้ไม่ใช่ลิสต์ ticket เก่า แต่เป็นรูปแบบ: 24 ใบ
เจียบเกิน 21 วัน · 22 ใบอยู่สถานะ Ready for test และวันที่ถูกแตะครั้งสุดท้ายกระจุกอยู่แค่ 2 วัน —
11 ใบวันเดียวกัน (เจียบ 57 วัน) และ 10 ใบอีกวัน (เจียบ 23 วัน) แปลว่างานถูกดันเข้า QA เป็นก้อน ๆ
แล้วไม่มีใครหยิบต่อ ก่อนเก่าค้างมา 8 สัปดาห์ โดยไม่มีข้อความใน Slack พูดถึงเลยแม้แต่คำเดียว
เกณฑ์ 21 วันมาจากจังหวะของตัวเอง ไม่ใช่ความรู้สึก — วันที่ ticket เปิดอยู่ไม่ถูกแตะมี $p_{25} = 5 \cdot$
 $\text{median} = 8 \cdot p_{75} = 22$ และ 21 ตกในช่องว่างธรรมชาติ (ไม่มี ticket ใบไหนอยู่ระหว่าง 14–21 วัน)
เกณฑ์จึงไม่ผ่ากลางกลุ่มหนาแน่นที่เลื่อนวันเดียวคำตอบเปลี่ยน

⚠ รายงานให้ตรง เชื้อไข “ไม่มีใครพูดถึงใน Slack” ไม่ได้ใช้เป็นตัวกรอง เพราะวัดแล้วมันไม่เปลี่ยน
อะไร (24 จาก 24 ที่เกณฑ์นี้) — ใส่เป็นตัวกรองจะทำให้ดูเหมือนต้องใช้สองแหล่งทั้งที่ไม่ใช่ · และ “ไม่มี
ใครพูดถึง” จำกัดอยู่ในหน้าต่าง 74 วันที่ export มา ไม่ใช่ทั้งประวัติ · ถ้าอ่าน ticket ไม่ได้

/api/tracker ส่ง error มาคู่กับลิสต์ว่าง และหน้าจอเขียนว่า “เรายังไม่รู้” ไม่ใช่ “ไม่มีงาน
ค้าง”

✓ ความครอบคลุม ตัวนี้ครอบคลุม 100% ของ ticket ที่เปิดค้าง เพราะไม่ต้องรอให้ Slack เอ่ยถึง
ต่างจากการเทียบสถานะที่ทำได้แค่ 13 จาก 38 work item ที่ผูกกับ ticket ได้ — และ drift ต้องมีคน
พิมพ์เลข ticket จริง ซึ่งเกิดขึ้นกับ 32 จาก 196 ใบ (16%) เท่านั้น

ข้อจำกัดที่เหลือ

drift หรือ “คู่ที่สถานะขัดกัน” ได้ แต่ยังมีพหุคูณ “ประโยคที่ตัดสินสถานะ” ไม่ได้ — แต่ละใบยืนอยู่บนสองประโยค: ประโยคที่มีคนพิมพ์เลข ticket (ทำให้เรื่องนี้ผูกกับ ticket ใบนั้น) และประโยคที่ทำให้สถานะฝั่ง Slack เป็นอย่างนั้น **วัดแล้วเป็นข้อความเดียวกัน 0 จาก 3 ใบ** — ใบ REVERAPP-87 อ้าง “additional fix [REVERAPP-110] and pending deploy to retest” ใต้หัวข้อที่พูดถึง REVERAPP-87 เพราะคลัสเตอร์กว้าง 45 ข้อความและประตูที่มีอยู่ตรวจแล้วว่า **มีใครพิมพ์เลขไว้ที่ไหนก็ได้ในเรื่องนี้** · ไม่ตัดใบพวกนี้ออก เพราะคลัสเตอร์ที่ผูกกับ ticket แล้วสถานะขัดกันยังควรมีคนไปดู แต่การดูทุกใบติดป้ายบอกไว้ และโซวสองประโยคให้เทียบกันเอง — วินัยเดียวกับป้าย `unverified` บนประโยคที่โมเดลเขียน · `/blockers` ยังรายงานต่ำกว่าความจริง แต่ช่องว่างแคบลงมาก — ตอนวัดรอบก่อนอ่านด้วยมือพบ blocker จริง 22 ข้อความ แต่รายงานได้ 2 · คอขวดถูกไล่ตัดด้วยการวัดจนเจอว่าเป็นระดับของสัญญาณ ไม่ใช่การจับกลุ่ม (blocked เท่ากันทุกค่า resolution 1.0–6.0) และไม่ใช้ลิสต์คำ (ลอง `pending` ที่โพสต์ 15 ครั้ง ได้เพิ่ม 1 แต่หลักฐานคือโพสต์ “Sprint 3 is officially end RECAP” จึงถอนออก) — ทีมเขียน blocker เป็น **บสรกัณฑ์**หนึ่งในโพสต์ daily ที่ยาวกว่าข้อความปกติ 12 เท่า (529 vs 46 ตัวอักษร · 14 จาก 15 เป็นโพสต์หลายบสรกัณฑ์) และมัก **ระบุตัวคนที่รออยู่** เช่น “insurance pending from mild” · **แก่ที่ระดับบสรกัณฑ์แล้ว** ทั้งฝั่ง blocker (`ingest/blockers.py`) และฝั่ง relation (`quoted.asserted_lines()`) ที่เก็บขอบบสรกัณฑ์ไว้) ผลคือหน้าต่าง 7 วันวันนี้รายงาน **blocked 7 จาก 38 งาน** ไม่ใช่ 2 — และ 58 คู่ที่ทวีตอ่านผิดถูกตัดออกโดยไม่สร้างของใหม่เลย · **reply ในเรดยังไม่ถูกนับ** corpus มี 1,492 record แต่ตัวโหลดดึง `slack_thread` ออก 191 อันตาม

ขยายไม่ได้ถ้าโตมาก

ค้นหาแบบ brute-force ไหวระดับหลายพันข้อความ ไม่ไหวระดับล้าน — ต้องมี vector database · `bge-m3` ใหญ่และช้ากว่าตัวเดิม รับผิดชอบ embedding ถูก cache ไว้แล้ว

default (`include_threads=False`)

เหลือ 1,301 — วัดครั้งล่าสุดบน 170 อัน
แรกพบ **110 อันเป็นเนื้อหาใหม่จริง** และมี
ข้อความขอความช่วยเหลือตรง ๆ อยู่ด้วย
เปิดใช้เป็นการแก้ flag เดียว แต่**ยังไม่ได้วัดว่า**
กระบวนการจับกลุ่มยังงี้ จึงไม่เปลี่ยนก่อนวัด
· ภาษาไทยไม่เว้นวรรค คำว่า **รอ** ไปเจอใน
“**รอบ**” “**กรอก**” จึงต้องตัดคำแล้วแมตซ์ระดับ
token

- 🐾 **ก้าวถัดไปที่ 1** หา cue ของ “ติด” จากข้อมูลจริง แล้ว**วัด precision ก่อนเชื่อ** — ไม่เติมคำแล้วคอม (คำ
อย่าง `blocker` 14 จาก 22 ครั้งมาจาก template standup ที่ขึ้นหัวว่า “Any blockers?” เติมเข้าไป
จะ fire ทุกโพลต์)
- 🐾 **ก้าวถัดไปที่ 2** ทำให้ drift พิสัย **ประโยคที่ตัดสินสถานะ** ได้ ไม่ใช่แค่ “มีใครพิมพ์เลขทิกเกิดไว้ที่ไหนก็ได้ใน
เรื่องนี้” — ประตู่ที่วัดไว้แล้วคือบังคับให้ anchor ต้องหาไม่ค่อยเจอใน corpus (0.7% เก็บ · 3.0% ตัด)
ยังไม่ลงเพราะหลักฐาน $n=3$ บางเกินไป
- 🐾 **ก้าวถัดไปที่ 3** ถ้าจะกลับมาเรนโมเดลอีก ต้องเปลี่ยน**ฐาน**เป็น `bge-m3` และหาข้อมูลมากกว่านี้หลาย
เท่า — เรนบน MiniLM ต่อด้วย corpus ระดับนี้ วัดแล้วว่าไม่คุ้ม

ตัวเลขทุกตัวบนสไลด์นี้มาจาก [docs/EXPERIMENTS.md](#)
— ตารางเต็มอยู่ที่นั่นทีเดียว สไลด์ยกมาแค่บรรทัดสรุป จะได้
ไม่มีสองแหล่งให้เพี้ยนกัน

วัดซ้ำได้ `npm run check-api`

ฝั่ง tech อ่านต่อได้ที่ [architecture.html](#) — flow 8 หัวข้อ · diagram 12 ภาพ · หลักการคิด 5 ข้อพร้อมการวัดที่
ทำให้ได้ข้อนั้น · และกราฟจากตัวเลขจริง 4 อัน

ตาม (Tam) · draft deck 10 สไลด์ · ปรับปรุง 19 ส.ค. 2569